How to explain this project in an interview

A script you can say out loud, with plain-language answers, scalability questions and the hard questions. Every number below is from the project's own logs. Where a number is still being refreshed, the guide says "a few percent" or "roughly" on purpose: say it the same way.

0. Why interviews go badly, and the fix (read this first)

Most "not convincing" interviews fail for three reasons, not for lack of knowledge:

1. **You start with the technology, not the problem.** The interviewer is lost by sentence two. Start with the problem, in one everyday picture.
2. **You list what you did, not what you found.** "I used Python, RAG, LLMs" sounds like a shopping list. "I found that X, and here is the number" sounds like an engineer.
3. **You hide the weak parts.** Interviewers always find them. If you say the limits first, calmly, you look senior and trustworthy.

The fix is one shape for every answer: **Problem -> What I did -> What I found (a number) -> What it cannot do.** Use it for every question in this guide.

1. The 30-second pitch (memorise this one exactly)

> *"Modern cars are computers on wheels, and the network inside them, called the CAN bus, has no passwords. Anyone who gets onto it can send fake messages, for example a fake 'the road is clear' to the braking system. I built a safe test bench that simulates a car's braking system and its network, attacks it in nine different ways, and measures one thing that usually gets ignored: not just whether an attack is detected, but whether it is stopped in time to avoid a crash. Then I added defences, proved how long an attacker can delay the brakes in the worst case, and checked the results against real attack recordings and a physics simulator."*

Stop there. Wait for the follow-up question. Do not keep talking.

2. The everyday picture (use this when the interviewer is not technical)

> *"Think of the car's network as an office where every employee believes every memo without checking who wrote it. The braking computer reads a memo from the distance sensor saying 'the road is clear'. An attacker slips in a forged memo saying that, or floods the office with so many memos that the real one arrives late. Either way the car brakes too late. My project asks: how late is too late, can we stop the forged memo, and can we prove the car still stops?"*

Three plain-language words you will use:

- **CAN bus**: the shared wire the car's computers use to talk to each other. It was designed in the 1980s with no security.
- **Point of no return**: the latest moment braking can still avoid the obstacle. After it, a crash is certain. It is a physics fact, not an opinion.
- **Detection deadline / margin**: how much time was left between noticing the attack and the point of no return. A detector that fires after the point of no return is useless, even if it is "accurate".

3. The 2-minute version (when they say "tell me about your project")

Problem. *"In-vehicle networks are insecure. Most research asks 'did the intrusion detector catch the attack?' and reports accuracy. But for a braking system the real question is 'was it caught early enough to avoid a crash?'. I couldn't find an existing framework that answers that on the bus level.*

What I built. *A simulator of the car's braking chain over a CAN bus, with realistic message queues and arbitration; nine attack types such as flooding the bus, delaying messages, or forging the sensor's distance; and detectors and defences. I added three defences: a second independent sensor, a gateway that blocks floods from untrusted ports, and message authentication with a safety watchdog.*

What I found. *The standard way of scoring detectors makes them look better than they are: the best one caught only about 6 in 10 dangerous attacks in time. A forged sensor that starts early is almost invisible to normal checks, but a second sensor removes it. I also computed a mathematical worst-case bound on how long an attacker can delay the brakes, and I tried to break my own bound with an automated search. 0 of 260 searches succeeded.*

How I checked it. *I replayed real attack recordings from a public dataset, and compared my braking model with a physics simulator called CARLA.*

What it cannot do. *It is emulation with assumed computer timings, and two attacker types still win: someone flooding from a device already on the safety bus, and someone who has stolen the signing key."*

4. The 5-minute walkthrough (for a technical round)

Say these in order. Each step is one or two sentences.

1. **The emulator.** "A discrete-event simulation in microseconds, so runs are exactly repeatable. CAN arbitration is modelled: the lowest ID wins, and nothing interrupts a frame in progress."
2. **The brake chain.** "Sensor -> perception -> decision -> brake actuator, over the bus. The ECU compute times are assumed; the bus delays are not assumed, they come out of queues and attacks."
3. **The plant.** "A closed-form braking model that gives the exact point of no return, so every run ends in 'crash' or 'safe stop' with a real number."
4. **Attacks.** "Nine families with declared strength limits, so a search can explore them fairly."
5. **Detection.** "Monitors learn normal behaviour from attack-free runs only and see only what is on the bus. I score them by the margin before the point of no return."
6. **Defences.** "Failover to a redundant radar; a gateway guard that whitelists messages from untrusted ports; authentication with a watchdog."
7. **The proof-style bound.** "Classic CAN response-time analysis plus a model of each attacker gives a worst-case latency. If it is below the point of no return, no random seed can cause a crash."
8. **Validation.** "Real data (13 attack recordings), a physics simulator (CARLA), a small C implementation of the decision logic, and adversarial search against my own bound."
9. **Engineering.** "484 automated tests, a CI pipeline with Docker, and a one-command reproduction that I run in the cloud to make sure results match on Windows and Linux."
10. **Limits.** "Assumed timings, idealised key management, two unstopped attackers."

5. What YOU did, and being honest about AI help

Interviewers will ask "what was your contribution?" Answer in three layers: **decisions, building, checking.**

> *"I set the direction: instead of scoring detectors by accuracy, score them by whether they act before the point of no return. I made the design decisions for the defences. I used an AI coding assistant heavily to implement and iterate, and I treated its output as untrusted: I ran it, I made it write tests, and I made it try to break its own claims. That caught real mistakes, for example an early result that looked too optimistic because it reused a handful of test scenarios."*

Rules:

- **Only say what is true for you.** If asked about any file, you must be able to explain it. Before the interview, open and read these four: the CAN bus emulator, the braking model, the brake chain, and the bound. Run the demo once yourself.
- **Never claim "I invented the detection-deadline idea".** It exists in earlier research. Say "I applied it to bus-level CAN attacks and added a provable bound".

6. Scalability questions, in plain language

Answer pattern: 1) say what scales and why, 2) say what you did NOT test, 3) say what you would do next. Interviewers respect the honest second part.

Q1. "How would this scale to a real car with hundreds of messages?"

> *"My test bench uses about a dozen message types, taken from a catalogue. The worst-case calculation walks through each message once per step, so its cost grows roughly in line with the number of messages, not with the square of it. The simulator also grows roughly in line with the number of frames. I have not tested hundreds of messages, so I would expect minutes rather than seconds, and I'd check that before claiming it."*

Q2. "How does it scale to thousands of attack scenarios?"

> *"Each scenario is independent, so they run in parallel. I already run the full set in the cloud on demand. I also use sampling with confidence intervals instead of trying every combination, and the mathematical bound covers a whole range of attacks at once, so I don't have to test each one."*

Q3. "Could this work for a fleet of vehicles?"

> *"It is a design-time tool, not a fleet monitor. For a fleet you would run the same analysis for each vehicle configuration, which is just more parallel jobs. Collecting data from cars on the road is a different system that I have not built."*

Q4. "What about newer networks like CAN-FD or Ethernet?"

> *"The structure carries over: message queueing, attackers, and a worst-case bound. The numbers and the arbitration rules change, especially for switched Ethernet, so the bound would need a new derivation. I would say it extends, but I haven't done it."*

Q5. "Would the defence run on a real small chip?"

> *"I wrote the decision logic in about 110 lines of simple C: no decimals, no memory allocation, no loops. It compiles to roughly 500 bytes of code and a 52-byte state on a PC. That says it is small enough to be plausible, but I have not run it on a microcontroller, so I don't claim real timing."*

Q6. "How do you scale the testing and the releases?"

> *"Every change triggers eight automated checks, including a Docker build. A separate workflow reruns every experiment in the cloud so I'm not tied to one laptop, and I compare it step by step with my local results. They matched on 16 of 19 steps; the rest were steps I had added since, plus one small bug in how I ordered results before summarising them, which I fixed."*

Q7. "You used an AI model. Won't it be slow or expensive at scale?"

> *"The AI is used for turning English requirements into machine-checkable rules, not in the safety path. I cache every response, so reruns cost nothing, and I designed it to work offline. The free-tier rate limits slowed one experiment, which I handled by making runs resumable."*

Q8. "How would you handle huge data, like long recordings?"

> *"The real dataset I used is about 3 GB. I convert it once to a compact format and cache it, then analyse with vectorised operations, so a full pass takes a couple of minutes. For terabytes I'd process in chunks and in parallel."*

Q9. "Where is the bottleneck?"

> *"The simulation itself, because each run steps through every bus event. I would speed it up by running scenarios in parallel, which is easy, and if needed by moving the core loop into a faster language."*

7. The hard questions (and honest answers)

"Isn't this just a simulation?"

> *"Partly, and I say so. That is why I added three outside checks: real attack recordings, a physics simulator for the braking model, and a stress test that tries to break my own bound. The simulator agrees with the physics engine to about 11 milliseconds once calibrated. My default braking numbers were more optimistic than that car, and I report it."*

"What is actually new?"

> *"Not the idea of a detection deadline. That exists. What I found no one had done is a worst-case brake-latency bound that depends on what the attacker can do, checked by trying to break it, plus a way to say which defence is the cheapest one that is enough for each attack. That is applied and incremental, and I describe it that way."*

"How do you know your bound is correct?"

> *"I can't prove it for a real car. For my model I tested it on tens of thousands of runs with zero violations, and I ran an automated adversary trying to push the real delay above the bound: 0 of 260 searches did. That adversary also caught three mistakes in my early versions, so I know it works."*

"What did you get wrong?"

> *"Two things I'm glad I caught. First, my early residual-risk number was too optimistic because I reused six test scenarios; with a fresh scenario for every sample it was a few percent instead of one or two. I published the correction. Second, a real-data experiment first detected 1 of 13 attacks because of a flawed filter; fixing it gave 13 of 13."*

"Why not just use machine learning for intrusion detection?"

> *"Machine learning scores are usually about accuracy per message. I care about whether the car brakes in time. I do include detectors, but I showed that the standard scores overstate protection, and that a simple second sensor or message signatures remove entire attack classes without any learning."*

"What would you do with more time?"

> *"Run the defence logic on real hardware, two cheap CAN boards on a real bus, and compare measured delays with my bound. Then extend to a second safety function and to newer network types."*

8. Behavioural answers (use the same shape)

- **Challenge:** "We had leaked access keys in old commits. I revoked them straight away and fixed the ignore rules so secrets stay out of the repository."
- **Teamwork:** "The repository had two other contributors. I merged my work through a pull request, with a merge commit, so their history stayed intact."
- **Failure:** "My first CI run on the new branch never ran, because the workflow only listened to old branch names. I found it by checking the run history instead of assuming it worked."
- **Learning:** "I learned to distrust results that look good: I now ask what would make this number wrong, and I write a test for it."

9. Delivery checklist

- Speak **slowly**. Pause after the pitch.
- Give **one number per answer**, not five.
- **Draw** the chain if there is a whiteboard: sensor -> perception -> decision -> brake, with the bus underneath and the attacker arrow.
- When you don't know: "I haven't tested that. My expectation is X because Y, and I would check it by Z."
- **Never** say "100% secure", "first in the world", or "better than every approach".
- End answers by **inviting a follow-up**: "I can go deeper on the bound or the defences, whichever you prefer."

10. One-page cheat sheet

Topic	Say
Problem	Car networks have no passwords; attackers can delay or forge brake messages
Key idea	Judge defences by whether they act before the point of no return
Built	Emulated car + 9 attacks + 3 defences + provable worst-case bound
Result 1	Standard detector scores overstate protection (best caught about 6 in 10 dangerous attacks in time)
Result 2	A second sensor removes forged-sensor attacks; message authentication plus a watchdog handles most of the rest
Result 3	Bound never violated; 0 of 260 adversarial searches broke it
Checks	Real attack data (13 recordings), physics simulator (about 11 ms), C port (138 of 138 cases)
Engineering	484 tests, 8-job CI, Docker, cloud reproduction

Topic	Say
Limits	Assumed timings; a flood from a device on the bus and a key thief still win; one safety function
Novelty	Applied and incremental: attack-aware bound plus defence assessment on top of the detection-deadline idea

Practice plan: day 1, memorise the 30-second pitch and the two-minute version; day 2, answer sections 6 and 7 out loud, recording yourself; day 3, do a mock interview with a friend who interrupts you. If you can explain the point of no return to someone who has never heard of CAN, you are ready.